IoT Weather Project

Measuring Forecast Accuracy for Seri Iskandar Without Hardware

[Your name] · [Student ID] · [Course code and name] · [Lecturer] · 8 November 2026

1. Introduction

Seri Iskandar, Perak, has a humid tropical climate in which rain often comes from short afternoon storms. Residents rely on forecasts for the town, yet those forecasts are rarely checked against what happened there. This project builds a complete IoT data pipeline that collects weather data, stores it in a cloud database, and measures how good the forecasts really are. No physical sensors were bought: the perception layer uses public weather services as virtual sensors, and every component runs on a free tier.

The objectives were to obtain at least three continuous days of weather data, to store forecasts beside actual values, to compute accuracy inside the database, to present the results on an interactive public dashboard, and to test whether machine learning can reduce forecast error.

2. Problem Statement

The project asks: **how accurate are weather forecasts for Seri Iskandar, which forecast model is best, how quickly does accuracy fall with lead time, and can the error be corrected?**

Data and duration. Sensors were not affordable for this assignment, so public archives are the data source. The report window is three full days, **Wed 4 Nov 00:00 to Sat 7 Nov 00:00 MYT (72 hours)**. It was retrieved from the archives after the event, not collected live. Live collection started on 7 Nov, and live data is kept outside the report window. A data check was fixed before any score was calculated: at least 65 of 72 complete hours, 6 rain hours and 60 paired hours per model. The window passed with 72 complete hours, 35 rain hours and 72 pairs per model, so the fallback of extending it to 1 Nov was not used.

What "actual" means. The primary actual is Open-Meteo's 15-minute analysis for the grid cell over the town [1]. It is model output, not a gauge. It is therefore cross-checked against METAR reports from Ipoh airport (WMKI), 26 km away, which come from real instruments [3].

Challenges. The archive keeps forecasts only at whole-day leads, so the window has no leads under 24 hours and no rain probability [2]. Its issue times are approximate, taken as the target hour minus N days. GitHub's scheduler can start late, so a daily job refills missed samples.

3. System Design

Fig. 1 maps the system to the five-layer IoT model. Every source is pulled over HTTPS, because none of them offers push delivery. Python jobs run on GitHub Actions, and each job is stateless and idempotent, so a repeated run adds no duplicates.

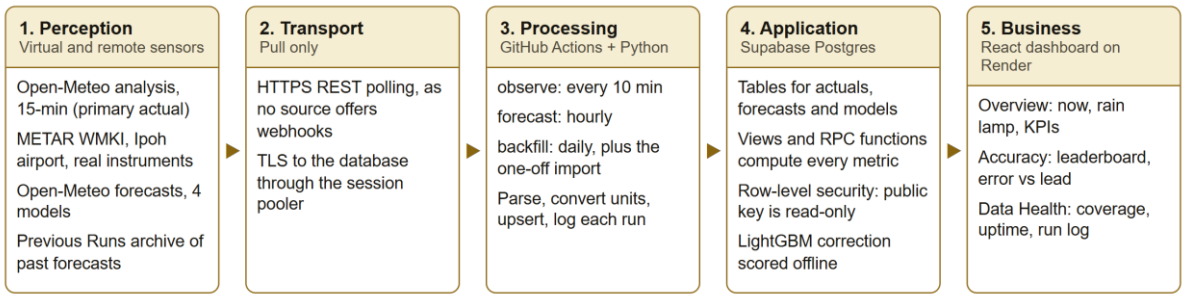


Fig. 1. System architecture mapped to the five-layer IoT model.

Data flow

- **Import path:** a one-off run of the backfill job loaded 1 to 7 Nov: 15-minute actuals, 144 METAR reports and 3,888 past forecast rows from four models.

- **Observation path:** every 10 minutes, observe stores the current analysis and new METAR reports.
- **Forecast path:** every hour, forecast stores each new model run once, checking run times so the same run is not counted twice.
- **Repair path:** every day, backfill refills missed samples and removes raw payloads older than 30 days.
- **Read path:** the browser calls database views and functions with a public key that can only read.

Times are stored in UTC, and local days are always converted explicitly to Asia/Kuala_Lumpur. Each run writes its steps, row counts and errors to a run log, which is shown on the dashboard. One failed source marks a run as partial and does not stop the others.

4. Database Design

The database is Supabase Postgres. Fig. 2 shows the main tables. PostgreSQL suits this project because every metric is a join between a forecast and an actual on the target hour, followed by aggregation, which SQL expresses directly and keeps next to the data.

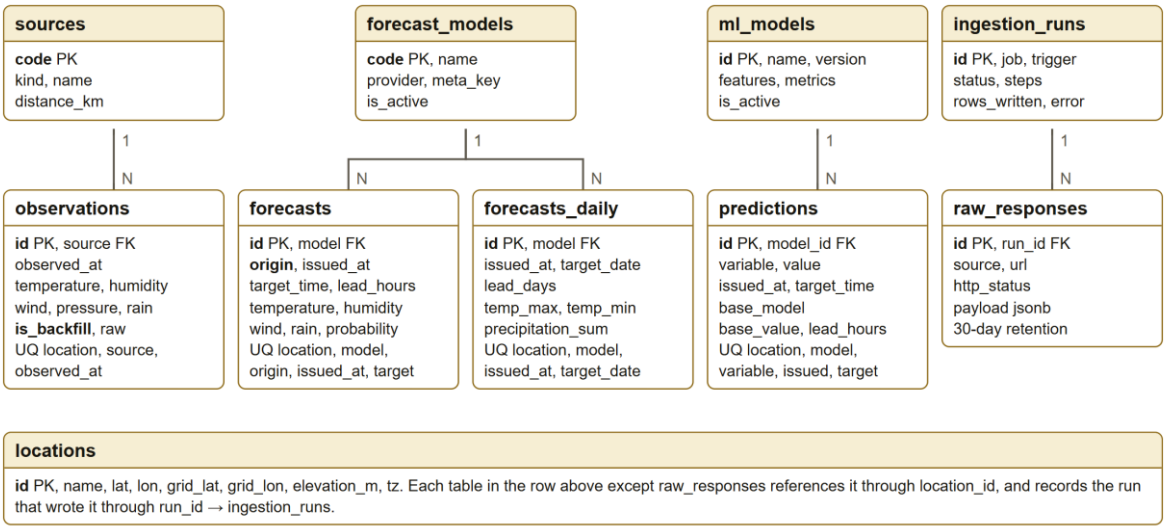


Fig. 2. Entity relationship diagram of the tables in use.

Keeping data honest. Two columns record where each row came from. The is_backfill column marks retrieved or repaired observations. The origin column separates live snapshots from retrieved forecasts. Because of this, the dashboard can show exactly when live collection began. Unique keys with ON CONFLICT make every write repeatable.

Analytics in the database. The view v_actual_hourly turns samples into hours, using the rule that hour H covers (H−1 h, H]. An hour counts only if all four samples exist. The view v_pairs joins forecasts to actuals in long format. Functions then compute MAE, bias, RMSE, POD, FAR and CSI [7], [8], the leaderboard and the next-day table, so the dashboard and this report read identical numbers. Row-level security gives the public key read access only [6].

Verification. 48 automated database tests check the SQL against hand-calculated results. A separate notebook recomputed the metrics in pandas from the raw tables, without using any view. MAE, bias and CSI matched the SQL to three decimal places for all 20 model and lead combinations, and the rain contingency table matched exactly.

5. Data Visualization

The dashboard is built with React and Apache ECharts [5] and hosted on Render at [dashboard URL]. It has three pages. All filters are stored in the URL, and it has light and dark themes and a phone layout.

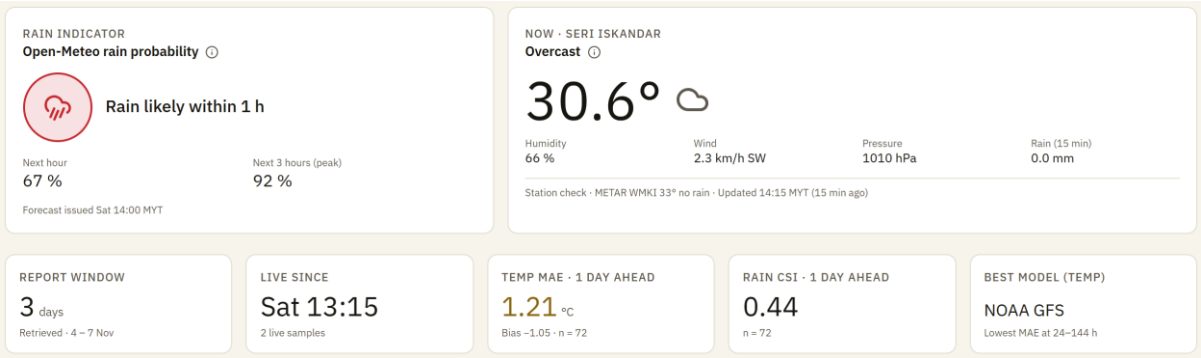


Fig. 3. Overview page: rain indicator, current conditions and headline scores.

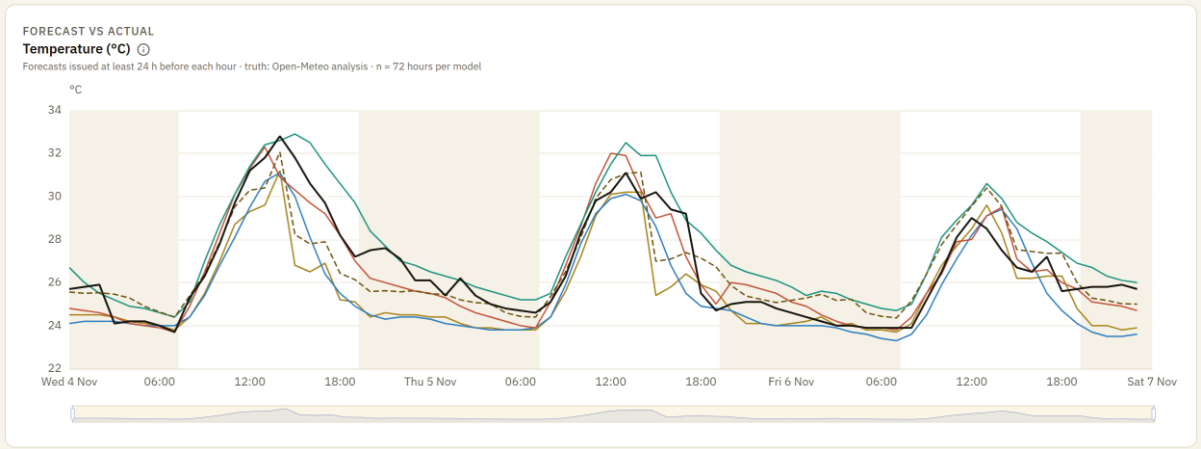


Fig. 4. Temperature forecast vs actual, with leads of at least 24 h. Night hours are shaded.

Fig. 4 shows the daily cycle clearly. Best Match errors are largest from noon to evening (MAE 1.9 °C from 12:00 to 18:00) and smallest in the morning (0.7 °C from 06:00 to 12:00). The user can change the variable, the models, the truth source and the lead. Fig. 5 shows that MAE generally rises with lead, and ranks the models.

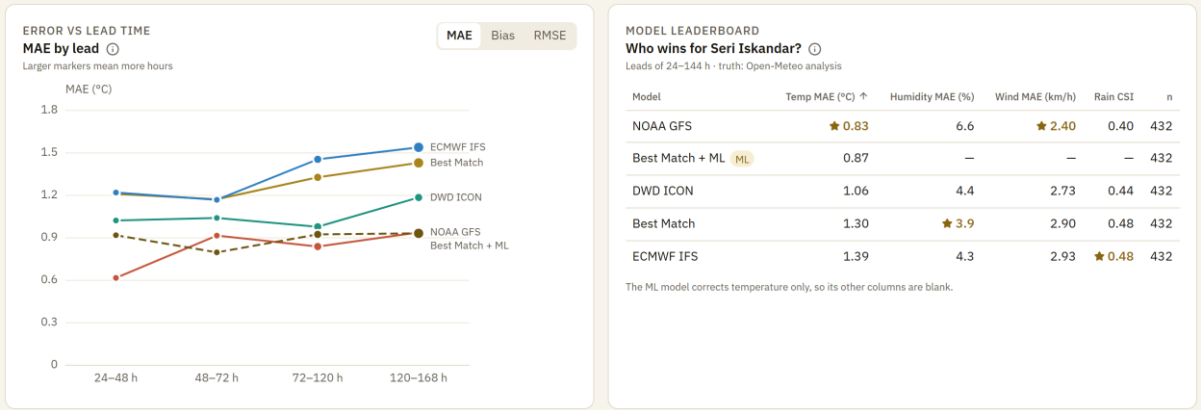


Fig. 5. Error by lead time, and the model leaderboard at leads of 24 to 144 h.

Table I. Model comparison, Wed 4 Nov to Sat 7 Nov, leads of 24 to 144 h, n = 432 per model

Model	Temp MAE (°C)	Humidity MAE (%)	Wind MAE (km/h)	Rain CSI
NOAA GFS	0.83	6.6	2.40	0.40
Best Match + ML	0.87	—	—	—
DWD ICON	1.06	4.4	2.73	0.44
Best Match	1.30	3.9	2.90	0.48
ECMWF IFS	1.39	4.3	2.93	0.48

The rain score uses a threshold of 0.1 mm per hour. No single model wins everything. GFS is best for temperature and wind, Best Match for humidity, and ECMWF by a small margin for

rain. At one day ahead, Best Match found 24 of 35 rain hours (POD 0.69), with a false alarm ratio of 0.44 and a CSI of 0.44. The brief's next-day table is built from hourly forecasts because no daily archive exists. It predicted the rain-day outcome correctly on all three days, and the daily maximum within 1.6 °C, though 6 Nov was over-forecast at 17.9 mm against 6.8 mm observed.

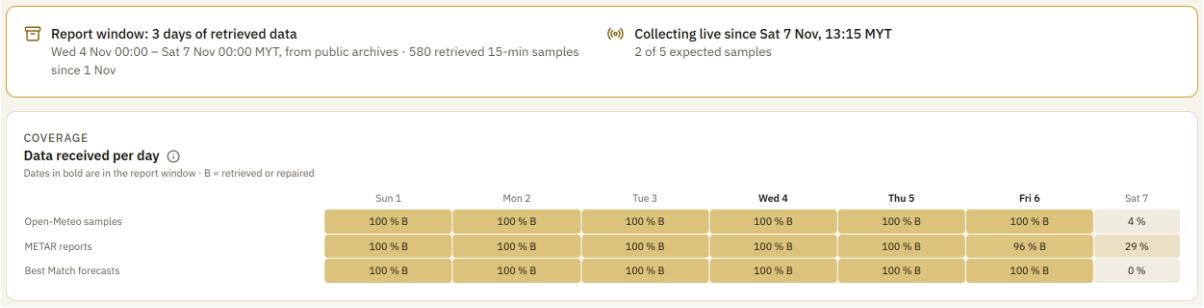


Fig. 6. Data Health page. "B" marks data retrieved from archives or repaired.

Machine learning: temperature bias correction

A LightGBM model [4] was trained to predict the error of the Best Match temperature forecast from the forecast itself, lead time, time of day, time of year, and forecast humidity, wind and rain. Inspection of the history found a clear change around Q3 2025: forecasts moved from about 1.3 °C too hot to between 0.5 and 1.0 °C too cold. Training on the full history gave a validation MAE of 1.217 °C, so the model was trained only on 1 Oct 2025 to 30 Jun 2026 (45,864 rows). On validation data from 1 Jul to 31 Oct 2026, it scored 0.987 °C, against 1.298 °C for the raw forecast and 1.025 °C for a constant correction. The model was then fixed and scored once on the report window, which it had never seen.

Table II. Test result on the report window, leads of 24 to 144 h, n = 432

Truth source	Raw MAE (°C)	ML MAE (°C)	Change	Raw bias (°C)	ML bias (°C)
Open-Meteo analysis	1.296	0.874	−32.6 %	−1.084	−0.043
METAR, real instruments	1.545	1.119	−27.6 %	−1.227	−0.186

6. Cost

The running cost is RM0, as Table III shows. The database uses 14 MB of its 500 MB allowance. A rough paid equivalent is about RM118 per month at RM4.06 to the US dollar: US$25 for Supabase Pro [10] and US$4 for a small server to run the jobs [11]. A commercial Open-Meteo plan would be extra, as the free tier is for non-commercial use only [9]. Its price could not be confirmed from the pricing page.

Table III. Components and free tiers used

Component	Free tier	Usage	Cost
Open-Meteo API	Non-commercial use, 300,000 calls a month [9]	About 7,500 calls a month	RM0
NOAA Aviation Weather	Public data	One call every 10 min	RM0
GitHub Actions	Free on public repositories	About 170 short runs a day	RM0
Supabase	500 MB database, 2 active projects [10]	14 MB, 1 project	RM0
Render	Free static site	1 site	RM0
Model training	Personal laptop	Minutes of computing time	RM0

7. Conclusion

The project delivered a working pipeline, a tested database and a public dashboard at no cost, and it answered its question. Over the report window, **NOAA GFS gave the best temperature forecast at 0.83 °C MAE**, while the rain scores of the models were close. Temperature error generally rose with lead. Best Match ran about 1.1 °C too cold. The bias correction removed

nearly all of that bias and cut MAE by 33 % against the analysis and 28 % against real instruments on unseen data, although GFS was still slightly better on its own.

Limitations. The window is short, covers one season, and was retrieved rather than collected live, so it has no leads under 24 hours and no Brier score. The primary truth is itself a model, and the correction was trained on similar data, which may flatter it, although the METAR result supports it. Rain is hard to verify: against the airport, 26 km away, CSI fell to between 0.09 and 0.17, which reflects how local storms are. The model covers temperature only, and it must be retrained if Best Match changes again. Live collection began on 7 Nov, with [uptime % from the Data Health page] scheduled uptime at the time of writing.

Future work. Longer live collection would add short leads and rain probability scores. Satellite imagery could support a rain nowcast, and a low-cost station at the town would replace the model-based truth.

References

[1] P. Zippenfenig, "Open-Meteo.com weather API," Zenodo, 2023. doi: 10.5281/zenodo.7970649.
[2] Open-Meteo, "Previous model runs API." Accessed: Nov. 7, 2026. [Online]. Available: https://open-meteo.com/en/docs/previous-runs-api
[3] NOAA Aviation Weather Center, "Data API." Accessed: Nov. 7, 2026. [Online]. Available: https://aviationweather.gov/data/api/
[4] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Proc. Adv. Neural Inf. Process. Syst., vol. 30, 2017, pp. 3146–3154.
[5] D. Li et al., "ECharts: A declarative framework for rapid construction of web-based visualization," Vis. Inform., vol. 2, no. 2, pp. 136–146, Jun. 2018.
[6] Supabase, "Row level security." Accessed: Nov. 7, 2026. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security
[7] D. S. Wilks, Statistical Methods in the Atmospheric Sciences, 4th ed. Amsterdam, The Netherlands: Elsevier, 2019.
[8] J. T. Schaefer, "The critical success index as an indicator of warning skill," Weather Forecast., vol. 5, no. 4, pp. 570–575, Dec. 1990.
[9] Open-Meteo, "Terms & privacy." Accessed: Nov. 7, 2026. [Online]. Available: https://open-meteo.com/en/terms
[10] Supabase, "Pricing & fees." Accessed: Nov. 7, 2026. [Online]. Available: https://supabase.com/pricing
[11] DigitalOcean, "Droplet pricing." Accessed: Nov. 7, 2026. [Online]. Available: https://www.digitalocean.com/pricing/droplets

Marking Rubric

Criterion	Marks	Where addressed	Awarded
Introduction	5	Section 1	
Problem statement	5	Section 2	
System and database design	30	Sections 3 and 4	
Data visualization	15	Section 5, dashboard	
Cost	5	Section 6	
Conclusion	5	Section 7	
Technical implementation	20	Source code, dashboard	
Report format	5	This report	
Presentation	10	Video	
Total	100		
Bonus (b): multi-model and lead-time metrics		Section 5	
Bonus (c): bias-correction ML, data health proof		Section 5	